Roteiro — Aula 06 (09-10): ORM com NestJS

PostgreSQL + TypeORM · partindo do `backend-final`

> Guia para escrever ao vivo. Partimos do `backend-final` (a API NestJS com o array em memória) e trocamos o array por um banco PostgreSQL via TypeORM.

Preparação: copie `aula_02-10` -> `aula_09-10` e trabalhe no backend dessa cópia.

---

1. Subir o PostgreSQL em container

Um banco roda melhor em container: isolado e com dados persistidos em um volume.

No `docker-compose.yml`, adicione o serviço `db` (ao lado do `frontend`):

```
  db:
    image: postgres:16-alpine
    container_name: web2-todo-db
    environment:
      POSTGRES_USER: todo
      POSTGRES_PASSWORD: todo
      POSTGRES_DB: todo
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

```
docker compose up -d db
```

---

2. Instalar e configurar o TypeORM

O ORM mapeia **classes <-> tabelas**, para falarmos com o banco em objetos, não em SQL.

```
npm install @nestjs/typeorm typeorm pg
```

Crie `src/data-source.ts` — as opções de conexão (usadas pela app) e o DataSource da CLI (que conhece as migrations):

```
import 'reflect-metadata';
import { DataSource, DataSourceOptions } from 'typeorm';
import { Tarefa } from './tarefas/entities/tarefa.entity';

export const dataSourceOptions: DataSourceOptions = {
  type: 'postgres',
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT) || 5432,
  username: process.env.DB_USER || 'todo',
  password: process.env.DB_PASS || 'todo',
  database: process.env.DB_NAME || 'todo',
  entities: [Tarefa],
  synchronize: false, // usamos migrations, não auto-sync
};

const dataSource = new DataSource({
  ...dataSourceOptions,
  migrations: ['src/migrations/*.ts'],
});
export default dataSource;
```

Em `src/app.module.ts`, conecte a aplicação:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { TarefasModule } from './tarefas/tarefas.module';
import { dataSourceOptions } from './data-source';

@Module({
  imports: [TypeOrmModule.forRoot(dataSourceOptions), TarefasModule],
})
export class AppModule {}
```

> Importante: a app usa `dataSourceOptions` (SEM `migrations`). Se as migrations `.ts` entrarem aqui, o app quebra ao iniciar. Elas ficam só no DataSource da CLI.

Adicione os scripts no `package.json`:

```
"typeorm": "typeorm-ts-node-commonjs",
"migration:generate": "npm run typeorm -- migration:generate -d src/data-source.ts",
"migration:run": "npm run typeorm -- migration:run -d src/data-source.ts",
"seed": "ts-node -r tsconfig-paths/register src/seed.ts"
```

---

3. A entidade vira uma tabela

Os decorators do TypeORM descrevem as colunas da tabela `tarefas`.

`src/tarefas/entities/tarefa.entity.ts`:

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity('tarefas')
export class Tarefa {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  titulo: string;

  @Column({ type: 'text', nullable: true })
  descricao?: string;

  @Column({ default: false })
  concluida: boolean;
}
```

---

4. Migrations

Migration = alteração **versionada** do banco (em vez de mudar a tabela na mão). O TypeORM compara a entidade com o banco e gera o SQL.

```
npm run migration:generate -- src/migrations/CreateTarefas
npm run migration:run
```

A migration gerada tem o `up` (aplica) e o `down` (desfaz):

```
export class CreateTarefas... implements MigrationInterface {
  public async up(q: QueryRunner): Promise&lt;void&gt; {
    await q.query(`CREATE TABLE "tarefas" (
      "id" SERIAL NOT NULL,
      "titulo" character varying NOT NULL,
      "descricao" text,
      "concluida" boolean NOT NULL DEFAULT false,
      CONSTRAINT "PK_tarefas" PRIMARY KEY ("id"))`);
  }
  public async down(q: QueryRunner): Promise&lt;void&gt; {
    await q.query(`DROP TABLE "tarefas"`);
  }
}
```

---

5. Seeder (dados iniciais)

Um script simples para popular a tabela. `src/seed.ts`:

```
import dataSource from './data-source';
import { Tarefa } from './tarefas/entities/tarefa.entity';

async function seed() {
  await dataSource.initialize();
  const repo = dataSource.getRepository(Tarefa);

  await repo.clear();
  await repo.save([
    { titulo: 'Estudar ORM', descricao: 'TypeORM + PostgreSQL', concluida: false },
    { titulo: 'Preparar a aula de hoje', concluida: true },
  ]);

  console.log('Seed concluído!');
  await dataSource.destroy();
}
seed();
```

```
npm run seed
```

---

6. Usar o banco no módulo de tarefas

Trocamos o array pelo **Repository** do TypeORM.

`src/tarefas/tarefas.module.ts` — registre a entidade:

```
import { TypeOrmModule } from '@nestjs/typeorm';
import { Tarefa } from './entities/tarefa.entity';
// ...
@Module({
  imports: [TypeOrmModule.forFeature([Tarefa])],
  controllers: [TarefasController],
  providers: [TarefasService],
})
export class TarefasModule {}
```

`src/tarefas/tarefas.service.ts` — injete o repositório e use seus métodos (agora tudo é assíncrono). O **controller não muda**.

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { CreateTarefaDto } from './dto/create-tarefa.dto';
import { UpdateTarefaDto } from './dto/update-tarefa.dto';
import { Tarefa } from './entities/tarefa.entity';

@Injectable()
export class TarefasService {
  constructor(
    @InjectRepository(Tarefa)
    private readonly tarefaRepo: Repository&lt;Tarefa&gt;,
  ) {}

  findAll(): Promise&lt;Tarefa[]&gt; {
    return this.tarefaRepo.find();
  }

  async findOne(id: number): Promise&lt;Tarefa&gt; {
    const tarefa = await this.tarefaRepo.findOneBy({ id });
    if (!tarefa) throw new NotFoundException('Tarefa não encontrada');
    return tarefa;
  }

  create(dto: CreateTarefaDto): Promise&lt;Tarefa&gt; {
    const tarefa = this.tarefaRepo.create(dto);
    return this.tarefaRepo.save(tarefa);
  }

  async update(id: number, dto: UpdateTarefaDto): Promise&lt;Tarefa&gt; {
    const tarefa = await this.findOne(id);
    Object.assign(tarefa, dto);
    return this.tarefaRepo.save(tarefa);
  }

  async remove(id: number): Promise&lt;void&gt; {
    const tarefa = await this.findOne(id);
    await this.tarefaRepo.remove(tarefa);
  }
}
```

---

7. Rodar e testar

```
docker compose up -d db        # banco
npm run migration:run          # cria a tabela
npm run seed                   # dados iniciais
npm run start:dev              # API em http://localhost:3000
```

Teste no **Postman** e no **frontend** (http://localhost:5173): CRUD normal. Agora **reinicie o backend** — os dados **permanecem** (estão no PostgreSQL, não mais em memória). A validação dos formulários continua funcionando igual.